

Neural Networks and Deep Learning Coursework Report

Module: ECS7026P Neural Networks and Deep Learning

Submission: Individual submission

Student name: Aaron Gulzar

Student number: 250933918

1. Introduction

This report presents the implementation and evaluation of neural network models for two image classification tasks. The first task focuses on CIFAR-10 classification using the required intermediate-block architecture from the assignment brief. The second task extends beyond CIFAR-10 by testing whether batch normalization improves the performance of a convolutional neural network on Fashion-MNIST.

For Part 1, I implemented a custom neural network where each intermediate block contains independent convolutional layers whose outputs are combined using weights generated from the average values of the input channels. The final selected CIFAR-10 model achieved a highest testing accuracy of 87.81%.

For Part 2, I carried out a small ablation study on Fashion-MNIST by comparing four CNN variants: a baseline CNN, a BatchNorm CNN, a BatchNorm CNN without dropout, and a BatchNorm CNN trained with data augmentation. The aim was to test how batch normalization, dropout, and data augmentation affected generalisation performance.

2. CIFAR-10 Image Classification

2.1 Architecture

The first part of the assignment required a neural network based on the intermediate-block architecture described in the assignment brief. The model was designed to classify CIFAR-10 images into 10 classes.

Each intermediate block contains several independent convolutional layers. These layers all receive the same input feature map rather than being connected sequentially inside the block. The input feature map is also averaged across its spatial dimensions to produce one mean value per channel. This vector is passed through a fully connected layer, which produces weights for combining the outputs of the independent convolutional layers.

A softmax operation was applied to these coefficients before combining the convolution outputs. This made the weights easier to interpret as relative contributions from each convolutional path and helped keep the weighted sum numerically stable. Each convolutional path also used batch normalization and ReLU activation to improve training stability and introduce non-linearity.

In my final model, four intermediate blocks were used. The number of output channels increased across the network:

Block 1: 3 input channels to 64 output channels

Block 2: 64 input channels to 128 output channels

Block 3: 128 input channels to 256 output channels

Block 4: 256 input channels to 256 output channels

Each intermediate block used four independent convolutional layers. Max pooling was applied between the first three blocks to reduce the spatial dimensions from 32×32 to 16×16 , then 8×8 , then 4×4 . The final feature map was passed to the output block, which averaged each channel and used fully connected layers to produce the final 10-class logits.

This design follows the required architecture because the convolutional outputs inside each intermediate block are combined using coefficients generated from the channel averages of the block input.

2.2 Training Setup

The CIFAR-10 dataset was loaded using PyTorch DataLoaders (Krizhevsky, 2009), with 50,000 training images and 10,000 testing images. The training data used random cropping and random horizontal flipping as data augmentation. Both the training and testing images were normalized using CIFAR-10 channel means and standard deviations.

The final selected model was trained for 30 epochs using cross entropy loss. AdamW was used as the optimiser with a learning rate of 0.001 and weight decay of 0.0005. A cosine annealing learning-rate scheduler was used during training. The batch size was 128.

AdamW was selected because it separates weight decay from the gradient update, making regularisation more controlled than standard Adam (Loshchilov and Hutter, 2019). Random cropping and horizontal flipping were used to reduce overfitting by exposing the model to small variations of the training images. The cosine scheduler gradually reduced the learning rate during training, allowing larger updates early on and smaller refinements later.

An initial model was trained first and achieved a highest testing accuracy of 84.76%. The improved model increased the number of channels and the number of independent convolutional layers per block. This improved the highest testing accuracy to 87.81%.

2.3 Results

The improved CIFAR-10 model achieved its highest testing accuracy of 87.81% at epoch 27. The loss curve in Figure 1 shows that the cross entropy loss decreased steadily during training. This indicates that the model was able to learn useful features from the CIFAR-10 training data.

Figure 2 shows the training and testing accuracy across 30 epochs. Training accuracy continued to increase and reached above 94%, while testing accuracy improved more slowly and peaked at 87.81%.

The gap between training and testing accuracy suggests some overfitting, but the testing performance remained stable near the end of training.

Table 1 summarises the improvement from the initial CIFAR-10 model to the improved model.

Model	Best testing accuracy
Initial model	84.76%
Improved model	87.81%

Table 1. CIFAR-10 model comparison. The improved model achieved the higher best testing accuracy, increasing performance from 84.76% to 87.81%.



Figure 1. Cross entropy loss for each training batch during training of the improved CIFAR-10 model. The loss decreases steadily across training, showing that the model learned from the training data.

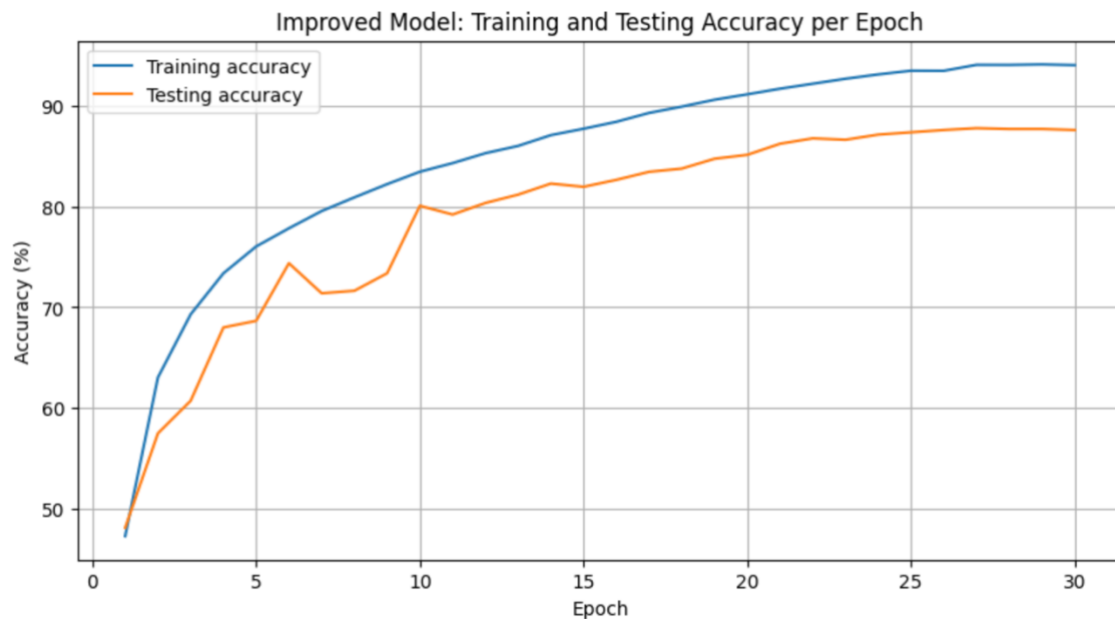


Figure 2. Training and testing accuracy per epoch for the improved CIFAR-10 model. The model achieved its highest testing accuracy of 87.81% at epoch 27. The gap between the training and testing curves suggests some overfitting.

3. Beyond CIFAR-10 Experiment

3.1 Dataset and Models

For the second part of the assignment, I used Fashion-MNIST as the dataset (Xiao, Rasul and Vollgraf, 2017). Fashion-MNIST contains 60,000 training images and 10,000 testing images of clothing items across 10 classes. The images are grayscale and have a size of 28×28 pixels.

The experiment was designed as a small ablation study to test how common CNN design choices affect Fashion-MNIST classification. Four model variants were compared. The first model was a baseline CNN without batch normalization. The second model added batch normalization after each convolutional layer and before the final hidden fully connected layer, following the method proposed by Ioffe and Szegedy (2015). The third model kept batch normalization but removed dropout, a regularisation method commonly used to reduce overfitting (Srivastava et al., 2014). The fourth model kept batch normalization and dropout but used data augmentation during training.

All four models used the same basic CNN structure, with two convolutional layers, max pooling, a fully connected hidden layer, and a final output layer with 10 logits. The main differences between the models were the use of batch normalization, dropout, and data augmentation.

Fashion-MNIST was chosen because it is more visually varied than digit classification while still being small enough to train quickly in Google Colab. This made it suitable for a controlled ablation study within the available computational budget.

3.2 Experimental Protocol

All models were trained using the same general experimental setup to keep the comparison fair. The batch size was 128 and each model was trained for 10 epochs. Cross entropy loss was used because the task is a multi-class classification problem. Adam was used as the optimiser with a learning rate of 0.001 (Kingma and Ba, 2015).

The baseline CNN, BatchNorm CNN, and BatchNorm without dropout model used the same Fashion-MNIST training and testing datasets. The augmented BatchNorm model used a second training DataLoader with random cropping and horizontal flipping, while still being evaluated on the same unchanged Fashion-MNIST testing dataset.

During each epoch, I recorded training loss, testing loss, training accuracy, and testing accuracy. I also recorded the training time for each model. The main comparison metric was best testing accuracy, because this measures how well each model generalised to unseen Fashion-MNIST images.

3.3 Results and Analysis

The updated experiment compared four CNN variants. The BatchNorm CNN achieved the strongest overall testing performance, with a best testing accuracy of 92.50%. The baseline CNN achieved 92.13%, the BatchNorm model without dropout achieved 92.05%, and the BatchNorm model with data augmentation achieved 92.00%. Table 2 summarises the final comparison between the four Fashion-MNIST models.

Figure 3 shows the testing accuracy curves for all four models. The BatchNorm CNN achieved the best overall result and remained one of the strongest models across training. This suggests that batch normalization improved optimisation and generalisation compared with the baseline CNN.

The model without dropout achieved very high training accuracy, reaching 98.82% by epoch 10. However, this did not translate into better testing accuracy. Its final testing accuracy dropped to 91.81%, suggesting that removing dropout increased overfitting.

The augmented BatchNorm model improved steadily and reached a final testing accuracy of 92.00%. However, it did not outperform the standard BatchNorm CNN within 10 epochs. It also took longer to train, with a training time of 194.51 seconds. This may be because random cropping and horizontal flipping made the training task harder, so the augmented model may need longer training to benefit fully from the extra variation.

Overall, the results suggest that batch normalization was the most useful change in this experiment. Dropout helped control overfitting, while data augmentation did not improve final performance within the limited 10-epoch training budget. A stronger conclusion would require repeated runs using different random seeds and longer training for the augmented model.

Model	Best test accuracy	Final test accuracy	Training time
Baseline CNN	92.13%	92.13%	138.92 s
BatchNorm CNN	92.50%	92.50%	140.98 s
BatchNorm, no Dropout	92.05%	91.81%	140.00 s
BatchNorm + Augmentation	92.00%	92.00%	194.51 s

Table 2. Fashion-MNIST ablation study results. The BatchNorm CNN achieved the best overall testing accuracy, while removing dropout increased overfitting and augmentation did not outperform the standard BatchNorm model within 10 epochs.

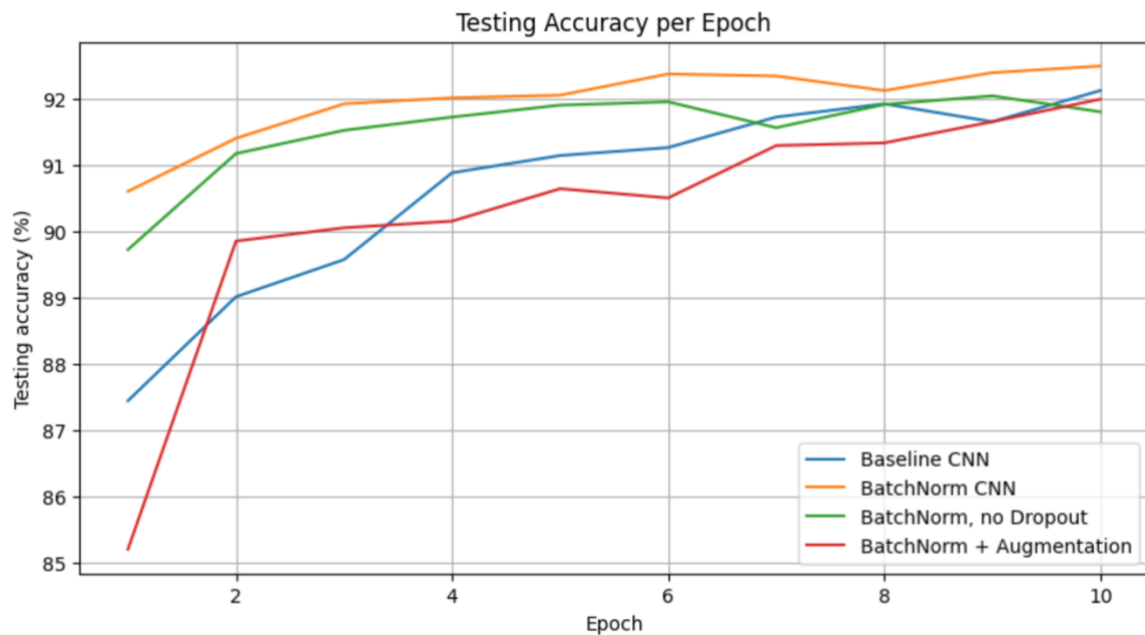


Figure 3. Testing accuracy per epoch for the Fashion-MNIST ablation study. The plot compares the baseline CNN, BatchNorm CNN, BatchNorm CNN without dropout, and BatchNorm CNN trained with data augmentation.

4. Limitations and Future Work

This work has several limitations. For Part 1, the CIFAR-10 testing dataset was used repeatedly while improving the model. This follows the assignment brief, but it would normally be better to use a separate validation set for model selection and keep the testing set only for final evaluation.

The final CIFAR-10 model also showed some overfitting because the training accuracy was higher than the testing accuracy. Further improvements could include stronger regularisation, longer hyperparameter search, or additional architecture changes that still preserve the required intermediate-block design.

For Part 2, each model was trained only once. This means the results may be affected by the random seed. Repeating the experiment several times and reporting mean accuracy and standard deviation would make the comparison more reliable. The augmented model may also need more than 10 epochs to show its full benefit. Future work could test deeper CNNs, longer training schedules, or stronger augmentation strategies.

5. References

- Ioffe, S. and Szegedy, C. (2015) 'Batch normalization: Accelerating deep network training by reducing internal covariate shift', Proceedings of the 32nd International Conference on Machine Learning, 37, pp. 448–456.
- Kingma, D.P. and Ba, J. (2015) 'Adam: A method for stochastic optimization', International Conference on Learning Representations.
- Krizhevsky, A. (2009) Learning multiple layers of features from tiny images. Technical report. University of Toronto.
- Loshchilov, I. and Hutter, F. (2019) 'Decoupled weight decay regularization', International Conference on Learning Representations.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014) 'Dropout: A simple way to prevent neural networks from overfitting', Journal of Machine Learning Research, 15, pp. 1929–1958.
- Xiao, H., Rasul, K. and Vollgraf, R. (2017) 'Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms', arXiv preprint arXiv:1708.07747.